

LUCY - BANKING ASSISTANT

BS COMPUTER SCIENCE

By

Youneeb Musa

Nouman Tariq

Supervisor: Dr.Mehreen Alam

Co-Supervisor: Mr.Pir Samiullah Shah

October 6, 2024



Contents

Contents	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Scope	1
1.3 Modules	2
1.3.1 Module 1: Customer Query Handling	2
1.3.2 Module 2: Escalation to human agent	2
1.4 User Classes and Characteristics	3
2 Project Requirements	5
2.1 Use Case/Event Response Table	5
2.1.1 Storyboard for Use Case: Check Account Balance	6
2.1.2 Storyboard for Use Case: Report Lost Card	7
2.2 Functional Requirements	7
2.2.1 Module 1: Customer Query Handling	7
2.2.2 Module 2: Escalation to Human Agent	9
2.3 Non-Functional Requirements	10
2.3.1 Reliability	10
2.3.2 Usability	10
2.3.3 Performance	10
2.3.4 Security	11
2.4 Domain Model for LUCY Banking Assistant	11
2.4.1 CustomerAttributes	12
2.4.2 LUCY Assistant (Bot)Attributes	12
2.4.3 Query Attributes	12
2.4.4 Customer Support AgentAttributes	13
2.4.5 Bank Attributes	13
2.4.6 Service Attributes	13
2.4.7 Language Model (Urdu NLP Model) Attributes	13
2.4.8 Feed Attributes	14

3 System Overview	15
3.1 ARCHITECTURAL DIAGRAM WITH DESCRIPTION	16
3.2 DESIGN MODELS	17
3.3 DATA REPRESENTATION	19
Bibliography	23

Chapter 1

Introduction

1.1 Problem Statement

In today's banking industry, customer service is the key to retain business. The existing customer service model needs to be upgraded according to the need of modern times; for instance, long waiting periods could be due to less staff or non-availability of urdu-literate staff. This causes disappointment among customers and they may reconsider their choice of bank. In addition, operational challenges are also a problem for bank. The project aims to overcome these inefficiencies by developing 'LUCY,' a virtual banking assistant that can resolve common banking queries in Urdu. By automating standard requests, such as checking account balances, or account opening information; LUCY will help eliminate language barriers, reduce the pressure on customer service representatives, and escalate complex issues to human agents only when necessary. This will enhance customer satisfaction while being cost-effective. Hence, improving bank efficiency.

1.2 Scope

The scope of the project is to develop a voice-based Urdu-speaking Virtual Assistant, LUCY, for the banking industry by automating frequent customer inquiries. LUCY will handle live-queries and reply to them to best of its knowledge. If something goes beyond its scope, the call may be to a human agent. The project aims to create a scalable SaaS solution capable of simultaneously managing large volumes of customer interactions. This will be achieved by utilizing NLP, integrated with banking systems for efficient query resolution. The project also aims to improve customer satisfaction by making banking services more accessible, particularly for Urdu-speaking customers, without increasing the workload on human service representatives.

1.3 Modules

Two significant modules have been identified in this project for handling customer service queries and escalating more complex requests. Each module plays a vital role in efficiently managing user interactions and ensuring a smooth flow within the banking support system.

1.3.1 Module 1: Customer Query Handling

This module primarily handles routine customer queries through interaction with a virtual assistant named LUCY. The Assistant communicates with customers via voice to process and respond to common banking inquiries, such as balance checks, account opening info, and card-related issues. By automating simple, repetitive tasks, the module reduces the workload on human agents and ensures prompt responses.

Features

1. Speech Command Capture and Recognition in Urdu

1. The system accurately captures and recognizes speech in Urdu using Natural Language Processing (NLP), allowing customers to communicate in their native language. This enhances the accessibility for Urdu-speaking customers
2. Tools Used: Google Cloud Speech-to-Text API, spaCy (or OpenNLP for natural language understanding). (tools may vary)

2. Real-time Response for Common Queries

1. LUCY provides immediate responses to frequently asked banking queries, such as balance inquiries, account opening info etc.
2. Tools Used: Machine learning models integrated with banking APIs to provide real-time responses.

3. Automated Resolution of Frequently Asked Questions (FAQs)

1. LUCY is designed to automatically identify and resolve the most frequently asked questions without human intervention, reducing the workload on human agents.

1.3.2 Module 2: Escalation to human agent

overview

In second phase, once assistant goes live then we will make sure to escalates the query to human agent if it is beyond the scope of LUCY. Furthermore, we will try to make the response time non-existent (instant responses).

Features

Improving response time:

1. Our model may take considerable time to respond back in urdu which will be sorted so lucy is able to give back instant replies.

Complex Query Detection and Escalation:

1. LUCY will detect when a query falls outside its pre-programmed capabilities. This could be due to the complexity of the query or the nature of the issue (e.g., disputes, errors, or legal matters). Once detected, the query is escalated to the human agent module, ensuring that the customer is not left without support.

This module ensures that customer satisfaction is maintained even when the virtual assistant is unable to resolve a problem. By providing a reliable escalation path, banks can still offer personal support for complicated queries, while relying on the virtual assistant to handle simpler, routine requests. This balance improves both efficiency and customer satisfaction.

1.4 User Classes and Characteristics

The project identifies several key user classes, each with distinct needs and characteristics:

1. **Banking Customers:**

The primary users of LUCY are customers, especially those who are Urdu-speaking and prefer native language interactions. They will use the system for routine banking queries such as balance checks and transaction histories. The virtual assistant's ability to provide quick, real-time responses in Urdu will improve customer satisfaction and make banking services more accessible to this demographic.

2. **Banking Institutions:**

The banks implementing LUCY will benefit from reduced operational costs and increased efficiency in customer service. The scalability of the system allows banks to handle a higher volume of customer interactions without increasing human resource requirements. This, in turn, leads to improved service levels and better customer retention.

3. **Customer Support Agents:**

Human agents will handle complex queries that LUCY cannot resolve. By automating routine queries, LUCY will reduce the workload on these agents, enabling them to focus on more critical and complex issues. This will lead to more efficient use of human resources and a higher standard of service for complex inquiries.

4. **Development and Operations Team:**

This group includes the developers, system administrators, and IT staff responsible for the creation, deployment, and maintenance of the LUCY virtual assistant. They will ensure that

the system operates efficiently, scales appropriately, and continues to meet the evolving needs of the banking sector.

Class Structure

Customer:

This class could represent a user interacting with LUCY. It may include attributes like:

- `customer_id`
- `name`
- `language_preference`
- `query_history`

Query:

This class could represent the queries made by customers. Attributes might include:

- `query_id`
- `customer_id` (linking to the Customer class)
- `query_text`
- `timestamp`
- `status` (e.g., resolved, pending, escalated)

Response:

This class could handle responses generated by LUCY. Attributes might include:

- `response_id`
- `query_id` (linking to the Query class)
- `response_text`
- `response_time`
- `language`

Class Methods

Each class can have methods to perform specific actions. For example, the Customer class might have methods like:

- `submit_query()`: To allow customers to submit their queries.
- `get_query_history()`: To retrieve previous interactions.

Chapter 2

Project Requirements

2.1 Use Case/Event Response Table

Use Case ID	Use Case Name	Actor	Preconditions	Trigger	Main Success Scenario	Postconditions
UC-01	Check Account Balance	Banking Customer	Customer has an account with the bank	Customer initiates a query	1. Customer says "Check my account balance." 2. LUCY processes the request and retrieves the balance. 3. LUCY responds with the current balance.	Customer receives the account balance.
UC-02	Report Lost Card	Banking Customer	Customer has a debit/credit card with the bank	Customer reports a lost card	1. Customer says "I lost my card." 2. LUCY confirms the request. 3. LUCY escalates the case to a human agent for further action.	The issue is escalated to customer support.

UC-03	Transaction History	Banking Customer	Customer has an account with transaction history	Customer wants to view transactions	1. Customer says "Show me my transaction history." 2. LUCY retrieves and displays the recent transactions.	Customer reviews their transaction history.
UC-04	Card Activation	Banking Customer	Customer has a new card to activate	Customer wants to activate a card	1. Customer says "Activate my new card." 2. LUCY processes the activation request. 3. LUCY confirms the card activation.	Card is activated successfully.
UC-05	Escalate Complex Query	Banking Customer	Customer has a query that cannot be resolved by LUCY	Customer presents a complex issue	1. Customer explains the issue. 2. LUCY recognizes it as complex. 3. LUCY transfers the call to a human agent.	The customer is connected to a human agent.

Storyboard

2.1.1 Storyboard for Use Case: Check Account Balance

1. Scene 1: User Initiation

- Visual: A user (banking customer) holds a smartphone.
- Text: "The user wants to check their account balance."

2. Scene 2: Voice Command

- Visual: Close-up of the user speaking into the phone.
- Text: "User: 'Check my account balance.'"

3. Scene 3: Processing Request

- Visual: A visual representation of LUCY processing the request (e.g., a loading icon).

- Text: "LUCY is processing your request..."

4. Scene 4: Response

- Visual: The user smiles as they read the response on the phone.
- Text: "LUCY: 'Your current balance is \$1,250.'"

5. Scene 5: User Satisfaction

- Visual: The user looks satisfied, perhaps nodding.
- Text: "The user is satisfied with the quick response."

2.1.2 Storyboard for Use Case: Report Lost Card

1. Scene 1: User Concern

- Visual: The user looks worried while holding their phone.
- Text: "The user realizes they lost their card."

2. Scene 2: Voice Command

- Visual: Close-up of the user speaking into the phone.
- Text: "User: 'I lost my card.'"

3. Scene 3: Confirmation

- Visual: LUCY's interface confirming the lost card report.
- Text: "LUCY: 'I can help you with that. Let me connect you to a human agent.'"

4. Scene 4: Escalation

- Visual: An animation of the call being transferred.
- Text: "Transferring to a customer support agent..."

5. Scene 5: Resolution

- Visual: The user speaks with a human agent on the phone.
- Text: "The customer support agent assists the user in blocking the card."

2.2 Functional Requirements

2.2.1 Module 1: Customer Query Handling

Voice Command Capture

1. The system must capture and process voice commands in Urdu.
2. The system should utilize a speech recognition API to convert voice input to text.

Query Recognition

1. The system should recognize predefined banking queries, such as:
 - label=. Checking account balance.
 - lbbel=. Retrieving transaction history.
 - lcbel=. Reporting lost or stolen cards.
 - ldbel=. Activating a card.
2. The system must also differentiate between simple and complex queries based on keywords.

Real-time Response Generation

1. The system should generate real-time response for recognized queries.
2. These responses should be clear and concise, providing relevant information to the customer.

User Feedback Mechanism

1. The system should allow users to provide feedback on the accuracy of the responses they receive.
2. User feedback should be logged to improve the system's performance.

Error Handling

1. The system must handle unrecognized commands gracefully, by prompting users to repeat or rephrase their queries.
2. The system should log instances of unrecognized queries for future training of the model.

Session Management

1. The system should maintain session context, allowing users to ask follow-up questions without repeating their initial requests.
2. The session should timeout after a predefined period of inactivity.

Solution Impact

- **Efficiency:** Automating responses to common queries reduces customer wait times, improving overall customer satisfaction.
- **Accessibility:** The assistant bridges the language gap for Urdu-speaking customers, providing them with an interface they are comfortable with for banking operations.
- **Scalability:** LUCY can become an international solution for global banks that face problem dealing Urdu-speaking customers.

2.2.2 Module 2: Escalation to Human Agent

Complex Query Detection

1. The system must identify complex queries that require human intervention based on specific triggers or patterns in user requests.
2. Queries should be analyzed for keywords and context to determine complexity.

Seamless Handoff Process

The system must provide a seamless transition to a human agent, sharing the user's context and previous interactions to avoid repetition.

Integration with Customer Support System

The system must integrate with the bank's existing customer support infrastructure to efficiently handle escalated queries. This should include logging details about the interaction for customer service representatives.

Notification System

The system should notify users when their query is being escalated to a human agent and provide an estimated wait time for a response.

Agent Assignment

The system should assign escalated queries to available human agents based on their expertise and availability. Agents should receive notifications about new escalated queries assigned to them.

Feedback Loop

After the escalation, the system should prompt users to rate their experience with the human agent. Feedback should be logged for continuous improvement of both the virtual assistant and human agent interactions.

Solution Impact

- **Improved Customer Experience:** By ensuring that complex queries are handled by human agents, customers receive the necessary support without feeling frustrated by the limitations of the virtual assistant.
- **Efficiency in Operations:** This module allows LUCY to handle a large volume of routine queries while ensuring that complex issues are escalated appropriately, reducing the overall workload on human agents.

- **Improved Training:** Data collected from escalated queries can be analyzed to improve LUCY's training, enabling the system to learn from complex interactions and enhance its handling of similar queries in the future.

Overall, Module 2 is essential for maintaining the balance between automated efficiency and the personalized touch of human customer service in the banking sector.

2.3 Non-Functional Requirements

2.3.1 Reliability

- **Uptime Requirement:** The system should guarantee at least 99.5% uptime, ensuring that customers experience consistent service availability.
- **Error Recovery:** The system must be equipped with mechanisms to recover quickly from failures or crashes, minimizing service disruption. Automatic recovery should start within 2 minutes.
- **Data Integrity:** The system must maintain the integrity of data during transactions, ensuring all transactions are recorded and verified without any data loss or corruption.

2.3.2 Usability

- **User Interface:** The interface should be simple and intuitive, specifically designed for Urdu-speaking customers to minimize the learning curve. Users should be able to find information within three clicks or fewer.
- **Accessibility:** The system must meet accessibility standards to be usable by individuals with disabilities, such as visual impairments, and include features like text-to-speech and speech-to-text functionality.
- **User Support:** Comprehensive help documentation should be available to guide users through common tasks and help troubleshoot issues without needing external assistance.

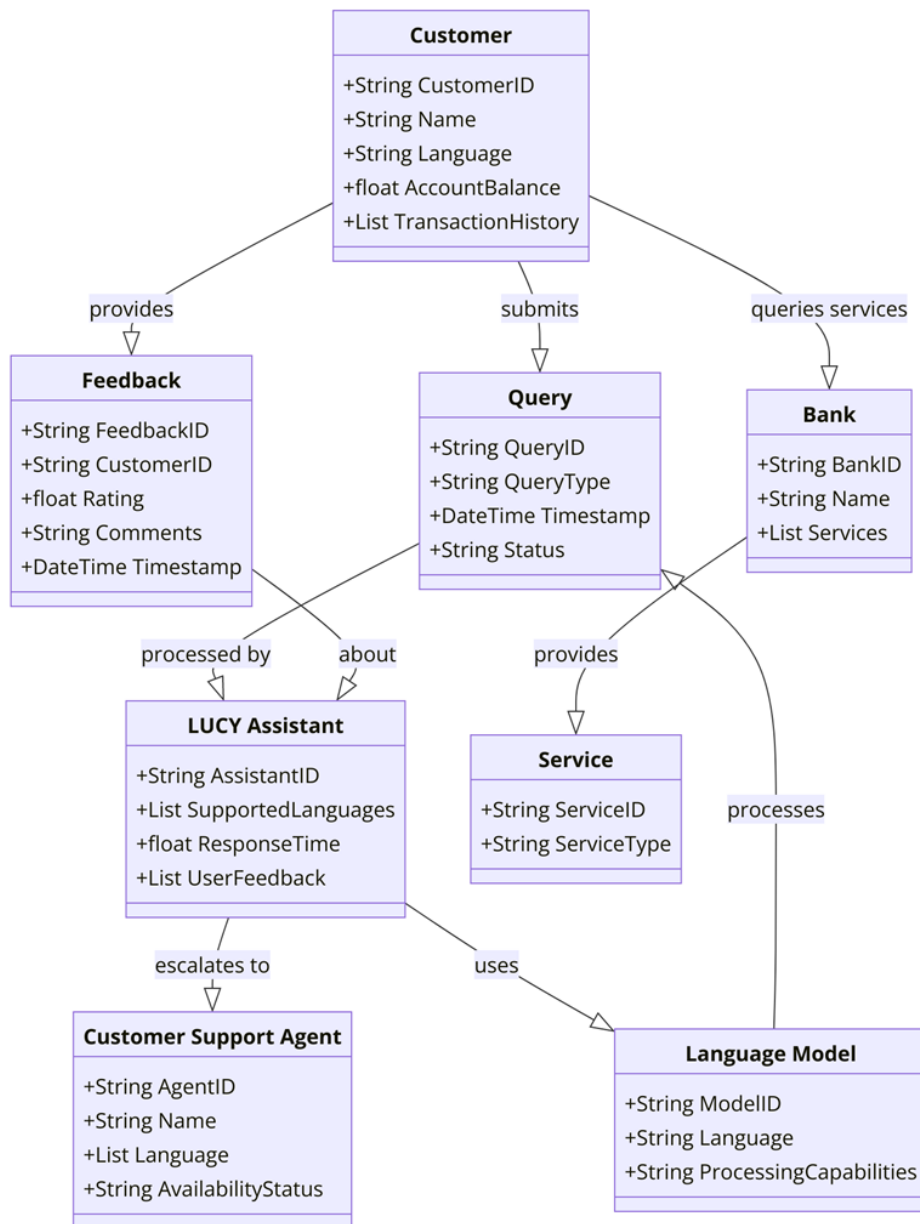
2.3.3 Performance

- **Response Time:** The system should respond to 95% of user queries within 3 seconds, ensuring a smooth user experience. This standard should also apply to voice recognition with maintained accuracy.
- **Scalability:** The system must be capable of handling up to 10,000 simultaneous users without any drop in performance, ensuring the infrastructure can scale with growing demand.
- **Load Testing:** The system must undergo rigorous load testing to maintain performance during peak traffic times, such as month-end banking activities.

2.3.4 Security

- **Authentication:** It will be implemented over helpline along with security measures banks already use. It is a helpline assistant so much sensitive information is not required. For such sensitive cases, call will be handed over to an agent.
- **Vulnerability Management:** Regular security audits should be conducted to identify and address vulnerabilities in the system. The system should adhere to OWASP security best practices to reduce exposure to common threats.

2.4 Domain Model for LUCY Banking Assistant



Domain Model Components:

2.4.1 CustomerAttributes

- CustomerID (Unique Identifier)
- Name
- Language (Preferred interaction language, e.g., Urdu)
- AccountBalance
- TransactionHistory (List of Transactions)

Relationships

- A customer can interact with the LUCY Assistant
- A customer may escalate queries to the Customer Support Agent.

2.4.2 LUCY Assistant (Bot)Attributes

- AssistantID (Unique Identifier)
- SupportedLanguages (List of supported languages)
- ResponseTime (Average time taken to respond to queries)
- UserFeedback (List of feedback provided by users)

Relationships

- LUCY Assistant handles queries from customers
- The assistant escalates complex queries to the Customer Support Agent if necessary

2.4.3 Query Attributes

- QueryID (Unique Identifier)
- QueryType (E.g., Balance Check, Transaction History)
- Timestamp (Time when the query was submitted)
- Status (E.g., Resolved, Escalated)

Relationships

- A query belongs to a Customer.
- A query is processed by the LUCY Assistant.
- A query may be escalated to the Customer Support Agent.

2.4.4 Customer Support Agent Attributes

- AgentID (Unique Identifier)
- Name
- Language (Languages spoken by the agent)
- AvailabilityStatus (E.g., Available, Busy, Offline)

Relationships

- Customer Support Agent handles escalated queries from the LUCY Assistant
- Provides customer support.

2.4.5 Bank Attributes

- BankID (Unique Identifier)
- Name
- Services (List of services provided by the bank, e.g., account management, card activation)
- AvailabilityStatus (E.g., Available, Busy, Offline)

Relationships

- The Bank provides multiple services to customers.
- These services can be queried by a Customer.

2.4.6 Service Attributes

- ServiceID (Unique Identifier)
- ServiceType (E.g., Balance Check, Card Activation)

Relationships

- A Service is available to the customer via queries processed by the LUCY Assistant.

2.4.7 Language Model (Urdu NLP Model) Attributes

- ModelID (Unique Identifier)
- Language (Language being processed, e.g., Urdu)
- ProcessingCapabilities (The capabilities of the model in handling natural language queries)

Relationships

- The Language Model processes customer queries in Urdu and provides the parsed result to the LUCY Assistant.

2.4.8 Feed Attributes

- FeedbackID (Unique Identifier)
- CustomerID (Identifier of the customer providing feedback)
- Rating (Numeric rating of the experience)
- Comments (Text comments about the experience)
- Timestamp (When the feedback was provided)

Relationships

- Feedback is related to both the Customer and the LUCY Assistant to capture the user experience
- Feedback may influence future system improvements

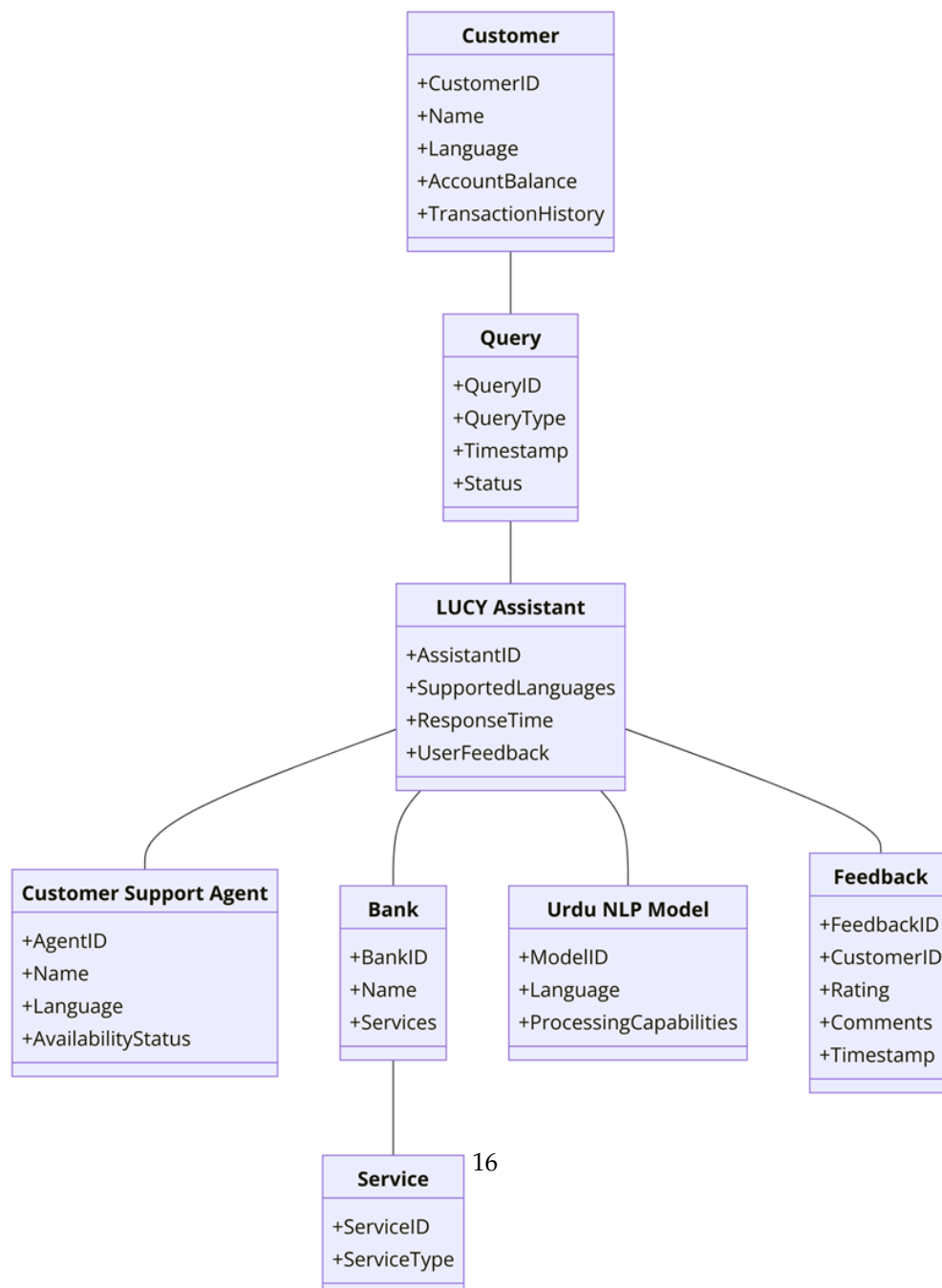
Entity Relationships

- Customer → Submits → Query → Processed by → LUCY Assistant
- LUCY Assistant → Escalates → Query → To → Customer Support Agent
- Customer → Queries → Bank Services → Provided by → Bank
- Customer → Provides → Feedback → On → LUCY Assistant
- LUCY Assistant → Uses → Language Model → To Process → Customer's natural language query

Chapter 3

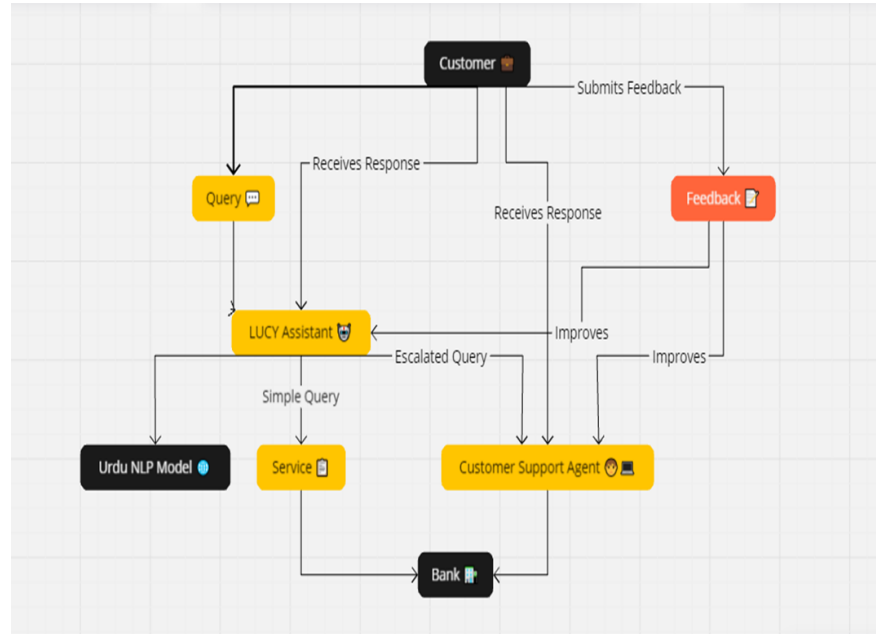
System Overview

3.1 ARCHITECTURAL DIAGRAM WITH DESCRIPTION

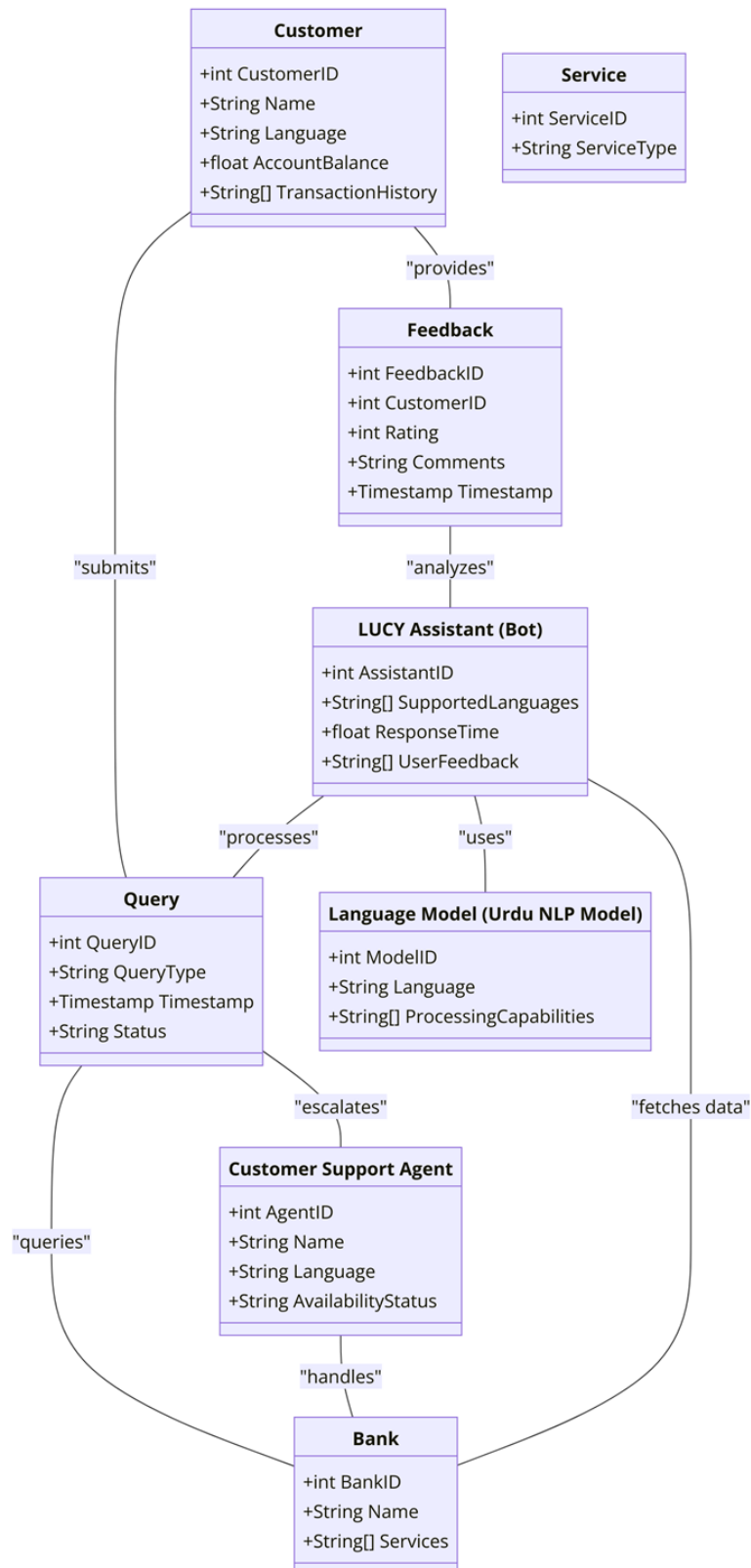


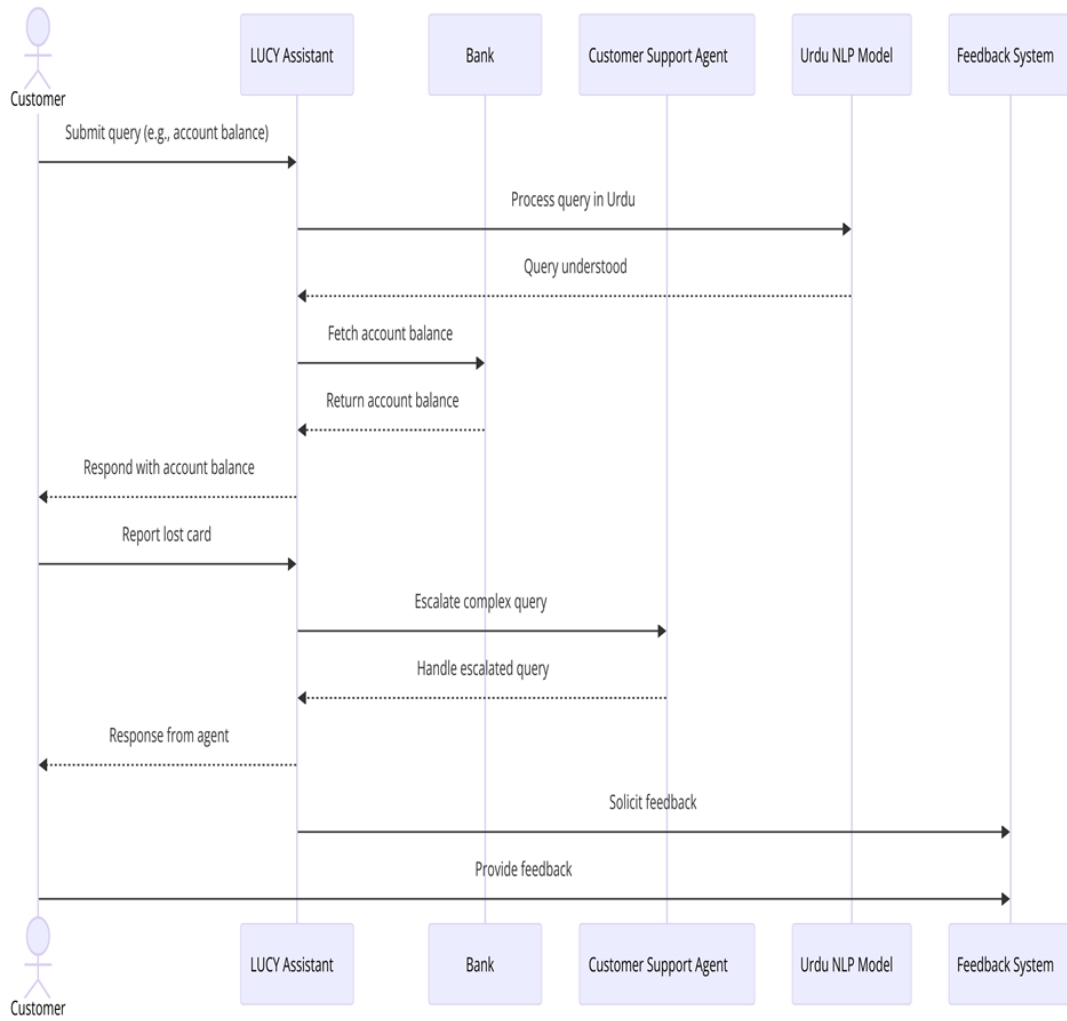
3.2 DESIGN MODELS

PROCESS FLOW(ACTIVITY DIAGRAM)



Class Diagram





3.3 DATA REPRESENTATION

```

1 {
2   "Customer": {
3     "type": "object",
4     "properties": {
5       "CustomerID": {"type": "string"},
6       "Name": {"type": "string"},
7       "Language": {"type": "string"},
8       "AccountBalance": {"type": "number"},
9       "TransactionHistory": {
10        "type": "array",
11        "items": {
12          "type": "object",
13          "properties": {
14            "TransactionID": {"type": "string"},

```

```

15         "Amount": {"type": "number"},
16         "Date": {"type": "string", "format": "date-time"}
17     }
18 }
19 }
20 },
21 "required": ["CustomerID", "Name", "Language"]
22 },
23
24 "LUCYAssistant": {
25     "type": "object",
26     "properties": {
27         "AssistantID": {"type": "string"},
28         "SupportedLanguages": {
29             "type": "array",
30             "items": {"type": "string"}
31         },
32         "ResponseTime": {"type": "number"},
33         "UserFeedback": {
34             "type": "array",
35             "items": {"$ref": "#/Feedback"}
36         }
37     },
38     "required": ["AssistantID", "SupportedLanguages", "ResponseTime"]
39 },
40
41 "Query": {
42     "type": "object",
43     "properties": {
44         "QueryID": {"type": "string"},
45         "QueryType": {"type": "string"},
46         "Timestamp": {"type": "string", "format": "date-time"},
47         "Status": {"type": "string", "enum": ["Pending", "Resolved", "Escalated"]}
48     },
49     "required": ["QueryID", "QueryType", "Timestamp", "Status"]
50 },
51
52 "CustomerSupportAgent": {
53     "type": "object",
54     "properties": {
55         "AgentID": {"type": "string"},
56         "Name": {"type": "string"},

```

```

57     "Language": {"type": "string"},
58     "AvailabilityStatus": {"type": "string", "enum": ["Available", "Unavailable"]}
59 },
60     "required": ["AgentID", "Name", "Language", "AvailabilityStatus"]
61 },
62
63     "Bank": {
64         "type": "object",
65         "properties": {
66             "BankID": {"type": "string"},
67             "Name": {"type": "string"},
68             "Services": {
69                 "type": "array",
70                 "items": {"$ref": "#/Service"}
71             }
72         },
73         "required": ["BankID", "Name"]
74     },
75
76     "Service": {
77         "type": "object",
78         "properties": {
79             "ServiceID": {"type": "string"},
80             "ServiceType": {
81                 "type": "string",
82                 "enum": ["Balance Check", "Card Activation", "Transaction History", "Report Lost
83                     ↪ Card"]
84             }
85         },
86         "required": ["ServiceID", "ServiceType"]
87     },
88
89     "LanguageModel": {
90         "type": "object",
91         "properties": {
92             "ModelID": {"type": "string"},
93             "Language": {"type": "string"},
94             "ProcessingCapabilities": {"type": "string"}
95         },
96         "required": ["ModelID", "Language", "ProcessingCapabilities"]
97     },

```

```
98  "Feedback": {
99    "type": "object",
100    "properties": {
101      "FeedbackID": {"type": "string"},
102      "CustomerID": {"type": "string"},
103      "Rating": {"type": "integer", "minimum": 1, "maximum": 5},
104      "Comments": {"type": "string"},
105      "Timestamp": {"type": "string", "format": "date-time"}
106    },
107    "required": ["FeedbackID", "CustomerID", "Rating", "Timestamp"]
108  }
109 }
```

Bibliography